

Tata Steel AI Hackathon - Mandatory Interview Preparation

1. Why didn't you deploy this online / provide a web link?

How to answer:

"This is a heavy, large-scale industrial application utilizing local LLM inference (Ollama) and computer vision models (Qwen-VL). Deploying these heavy models to the cloud would be incredibly expensive and defeat the purpose of the architecture which is Data Privacy. Tata Steel's data shouldn't be exposed to public web servers. The prototype runs locally because in production, it is meant to run on Tata Steel's secure, internal edge servers on the factory floor, ensuring zero data leakage and offline availability."

2. What makes your idea different from the other teams who used API services (OpenAI, Gemini)?

How to answer:

"Many teams took the easy route of sending data to OpenAI. That is an absolute dealbreaker for a multi-billion dollar industrial company like Tata Steel due to intellectual property risks. My architecture is 100% open-source and local. It proves that Tata Steel can have ChatGPT-level intelligence inside their own firewall without paying API fees or risking data security."

3. Your video was 23 minutes long. Can you give us the 2-minute pitch?

How to answer:

"My solution is a 3-tier Industrial AI Assistant. It features a Multimodal Vision model to diagnose broken equipment from photos, a local RAG pipeline to instantly search safety manuals, and a conversational agent that automatically generates inventory and work orders for supervisors. It brings the entire factory workflow into one secure, local interface."

4. What was the hardest technical challenge you faced?

How to answer:

"Orchestrating multiple local AI models without crashing the system. I had to build an intent router so the system knew when to use the Vision model vs. the RAG model, and I had to implement lazy-loading so the heavy models wouldn't consume all the system RAM at startup."

5. How do you handle large PDF manuals?

How to answer:

"Heavy manuals are processed asynchronously. They are chunked with a 100-character overlap to ensure context isn't lost between pages, and stored efficiently in the vector database."

6. How would you scale this for all Tata Steel plants globally?

How to answer:

"I would migrate the SQLite database to a distributed PostgreSQL cluster. The FastAPI backend is completely stateless, meaning it can be containerized using Docker and deployed on Kubernetes. The local

Tata Steel AI Hackathon - Mandatory Interview Preparation

LLMs could be hosted on dedicated internal GPU inference servers accessible via private endpoints by the different plants."

7. How do you prevent the AI from giving dangerous or wrong safety advice (Hallucinations)?

How to answer:

"I implemented Strict Contextual Grounding via Prompt Engineering. The AI is strictly instructed to only answer based on the retrieved ChromaDB vector context from the official Tata SOPs. If the answer isn't in the manual, it refuses to guess and tells the engineer to escalate to a supervisor."

8. Why did you separate the dashboard into Manager, Supervisor, and Engineer portals?

How to answer:

"To reflect real industrial workflows and maintain security. Engineers shouldn't be able to alter core inventory counts, and managers shouldn't be diagnosing broken pipes. Engineers log the issues, the AI translates those into requests, Supervisors approve the requests and budget, and Managers oversee the global data/documents."

9. How does the Vision Model actually detect the defects?

How to answer:

"The system uses Hugging Face's Qwen3-VL multimodal architecture. When an image is uploaded, it is base64 encoded and passed to the Vision Service. The model analyzes the pixels alongside a specialized prompt, allowing it to output a structured JSON identifying the equipment type, specific defects like corrosion, and a calculated risk level."

10. How does the Chatbot connect to the SQLite Inventory Database automatically?

How to answer:

"I built an autonomous intent router. If the Engineer asks for a spare part, the LLM generates a hidden system trigger called [INVENTORY_REQUEST]. My FastAPI backend intercepts this string, extracts the part name, queries the SQLite inventory database, deducts the stock, and logs an approval request for the Supervisor."